

45 .NET Web API INTERVIEW QUESTIONS

Production Scenarios, Trade-offs & Red Flags (2026 Edition)

Target Stack: .NET 10 / C# 14
Pillars: 9 Core Topics
Level: Jr / Mid / Sr

Overview & Purpose: This study guide compiles 45 scenario-based interview questions that evaluate real-world production experience in modern ASP.NET Core Web API development. Rather than focusing on simple definitions, each question tests practical architectural judgment, framework internals, trade-off analysis, and security best practices.

1. API DESIGN & REST BEST PRACTICES

6 Questions

Q1. Your API returns a list of products. A client asks for pagination. How do you implement it?

[Senior]

Great Answer Strategy:

- Default to offset-based pagination (`?page=1&pageSize=20`) with metadata (`totalCount`, `totalPages`, `hasNextPage`).
- For large/high-volume tables, switch to cursor-based (keyset) pagination (`?cursor=abc123`) to avoid expensive `COUNT(*)` queries and skip offsets.
- Enforce server-side max page size (e.g. 50) to prevent denial-of-service query payloads.

Red Flag Answer: Simply using `.Skip()` and `.Take()` in EF Core without considering performance or response envelope metadata.

Follow-up Question: "What happens to offset pagination when a record is deleted between requests?"

Q2. Your POST endpoint creates a resource but triggers an async background job. What status code do you return?

[Mid]

Great Answer Strategy:

- If resource creation is synchronous and job (e.g. email) is a side effect -> Return 201 Created with Location header.
- If the resource creation itself is queued/deferred -> Return 202 Accepted with a status tracking endpoint URL in the Location header.

Red Flag Answer: Returning 200 OK for resource creation endpoints.

Follow-up Question: "How should the status polling endpoint respond when the background task fails?"

Q3. A client sends a PATCH request to update only the email field. How do you implement partial updates?

[Senior]

Great Answer Strategy:

- Option 1: `JsonPatchDocument` (RFC 6902) for official JSON Patch standards.
- Option 2: Nullable DTOs tracking explicitly sent request fields (distinguishing null vs omitted).
- Option 3: Use PUT with full resource replacement for internal APIs where entities are compact.

Red Flag Answer: Using standard DTOs without handling the null vs missing property distinction.

Follow-up Question: "When does PATCH add unnecessary complexity compared to PUT?"

Q4. What's the difference between Query String parameters and Route parameters?

[Junior]

Great Answer Strategy:

- Route parameters identify a specific resource (/api/products/42) and are required.
- Query parameters modify/filter/sort/page the response (/api/products?category=tech) and are optional.

Red Flag Answer: Claiming they are identical in purpose, or placing resource identifiers in query strings like /api/products?id=42.

Follow-up Question: "How does ASP.NET Core route matching handle constraint validation?"

Q5. Your API accepts JSON but you want to enforce a max request size of 1MB. How?

[Senior]

Great Answer Strategy:

- Configure Kestrel globally: `options.Limits.MaxRequestBodySize = 1_048_576`.
- Use `[RequestSizeLimit(1_048_576)]` attribute for endpoint-specific overrides.
- Ensure reverse proxy limits (Nginx, Azure App Gateway) match Kestrel limits.
- Use Exception Handling Middleware to return `ProblemDetails` for HTTP 413 Payload Too Large.

Red Flag Answer: Unaware that Kestrel defaults to ~28.6MB and trusting default client payloads.

Follow-up Question: "How do you stream file uploads exceeding memory limits without buffering?"

Q6. What's the difference between [FromBody], [FromQuery], [FromRoute], and [FromHeader]?

[Junior]

Great Answer Strategy:

- [FromRoute] binds path parameters, [FromQuery] binds query string, [FromBody] parses JSON body, [FromHeader] reads headers.
- ASP.NET Core infers sources (simple = query/route, complex = body), but explicit binding avoids misinterpretation.
- Crucial rule: Only ONE [FromBody] parameter is allowed per action endpoint.

Red Flag Answer: Relying purely on auto-inference without knowing its edge-case failures.

Follow-up Question: "How do you bind multiple complex types in a single endpoint?"

2. ASP.NET CORE INTERNALS & MIDDLEWARE

6 Questions

Q7. You need Request Logging, Auth, Rate Limiting, CORS, and Exception Handling. What order?

[Mid]

Great Answer Strategy:

1. UseExceptionHandler (catches all subsequent exceptions)
2. UseHsts & UseHttpsRedirection (security layer)
3. UseCors (must precede auth so OPTIONS preflight requests aren't challenged with 401)
4. UseRateLimiter (blocks DDoS before expensive auth execution)
5. UseAuthentication & UseAuthorization
6. Request Logging (logs user identity after auth), 7. MapControllers/Endpoints.

Red Flag Answer: Registering CORS after Authentication, causing preflight request failures.

Follow-up Question: "What happens when middleware calls `next()` after writing to `Response.Body`?"

Q8. You have a Scoped service injected into a Singleton. What happens?

[Senior]

Great Answer Strategy:

- Captive Dependency Problem: The scoped service is captured by singleton and lives forever.
- Disposal never happens; DbContext captive dependencies cause thread safety bugs, stale tracking, and connection leaks.
- Detection: Enable ValidateScopes = true and ValidateOnBuild = true in Host configuration.
- Fix: Inject IServiceScopeFactory into singleton and create a scope per execution.

Red Flag Answer: Suggesting making the scoped service a Singleton to 'fix' the error.

Follow-up Question: "How does IHttpClientFactory prevent handler lifetime issues?"

Q9. What's the difference between Middleware and Endpoint Filters?

[Mid]

Great Answer Strategy:

- Middleware executes on every HTTP request before routing; works with raw HttpContext.
- Endpoint Filters execute only on matched endpoints and have typed access to arguments and return results.
- Use Middleware for pipeline concerns (logging, CORS, rate limits); use Filters for endpoint validation/authorization.

Red Flag Answer: Thinking Middleware and Endpoint Filters are interchangeable.

Follow-up Question: "How do Endpoint Filters compare with classic Action Filters in Controllers?"

Q10. What happens when you call AddScoped twice with different implementations?

[Mid]

Great Answer Strategy:

- The last registered implementation wins when resolving single interface IService.
- Both registrations remain in container; resolving IEnumerable returns both implementations.
- Use TryAddScoped to prevent overwriting existing registrations (essential for library creators).

Red Flag Answer: Expecting the container to throw a duplicate registration exception.

Follow-up Question: "How does the Decorator pattern utilize multi-registration in DI?"

Q11. How would you implement rate limiting in a .NET API?

[Mid]

Great Answer Strategy:

- Use built-in Microsoft.AspNetCore.RateLimiting middleware (.NET 7+).
- Select algorithm: Sliding Window (smooths bursts), Fixed Window, Token Bucket (burst allowed), or Concurrency Limiter.
- Apply globally or per-endpoint using .RequireRateLimiting('policyName').

Red Flag Answer: Creating a custom in-memory Dictionary rate limiter without thread safety or distributed state handling.

Follow-up Question: "How do you implement distributed rate limiting across multiple server instances?"

Q12. How does the request pipeline differ behind a reverse proxy?

[Senior]

Great Answer Strategy:

- Proxy terminates TLS/IP info; HttpContext loses client IP, scheme (HTTPS), and host header.
- Fix: Configure ForwardedHeadersOptions (X-Forwarded-For, X-Forwarded-Proto) and call UseForwardedHeaders() early in pipeline.
- Specify KnownProxies to prevent IP spoofing attacks.

Red Flag Answer: Exposing Kestrel directly to the internet without a reverse proxy.

Follow-up Question: "Why can misconfigured forwarded headers lead to infinite HTTPS redirect loops?"

Q13. Your API returns 200 products but SQL Profiler shows 201 queries. What happened?

[Senior]

Great Answer Strategy:

- N+1 Query Problem: 1 query gets products, then 200 lazy-loaded queries fetch category navigation properties.
- Fix 1: Eager loading via `.Include(p => p.Category)` [1 JOIN query].
- Fix 2: DTO Projection via `.Select(p => new ProductDto { ... })` [optimal, fetches only needed columns].
- Fix 3: Split query `.AsSplitQuery()` when multiple Includes cause cartesian explosion.

Red Flag Answer: Disabling lazy loading without adding Eager Loading or Projections.

Follow-up Question: "When should you prefer `.AsSplitQuery()` over single JOIN queries?"

Q14. How do you run EF Core migrations in production CI/CD?

[Mid]

Great Answer Strategy:

- NEVER call `db.Database.Migrate()` on app startup in production (risk of multi-instance race conditions and locks).
- Generate idempotent SQL scripts via `dotnet ef migrations script --idempotent`.
- Apply SQL scripts as a dedicated database deployment step in the CI/CD pipeline prior to app deployment.

Red Flag Answer: Executing automatic migrations on `Program.cs` startup in production environments.

Follow-up Question: "How do you handle zero-downtime database schema changes with breaking column renames?"

Q15. Two users submit conflicting price updates simultaneously. How do you handle this?

[Senior]

Great Answer Strategy:

- Implement Optimistic Concurrency Control using a `[Timestamp] byte[] RowVersion` property on entity.
- EF Core adds `RowVersion` to SQL WHERE clause; if altered, zero rows update and `DbUpdateConcurrencyException` is thrown.
- Catch exception and return 409 Conflict with `ProblemDetails` instructing client to refresh data.

Red Flag Answer: Allowing 'Last Write Wins' without concurrency detection, causing accidental data overwrites.

Follow-up Question: "How do you pass the `RowVersion` byte array through JSON REST DTOs?"

Q16. What's the difference between `AsNoTracking()` and default tracking?

[Mid]

Great Answer Strategy:

- Default tracking keeps entity snapshots in memory for change detection on `SaveChangesAsync()`.
- `AsNoTracking()` bypasses tracking overhead, saving memory and CPU; ideal for read-only GET endpoints.
- Can set `QueryTrackingBehavior.NoTracking` as default in `DbContext` options and explicitly opt-in for updates.

Red Flag Answer: Using `AsNoTracking()` on queries where you intend to modify and save entity changes.

Follow-up Question: "What is `AsNoTrackingWithIdentityResolution()` and when is it required?"

Q17. Your EF Core query works with 100 records but times out with 100,000. How do you fix it?

[Senior]

Great Answer Strategy:

- Extract generated SQL using `query.ToString()` and analyze execution plan in database tool.
- Common fixes: Add missing SQL indexes, use `.Select()` projection, apply pagination (`.Skip/.Take`), avoid client evaluation.
- Use `.AsSplitQuery()` for heavy JOINS or `EF.CompileAsyncQuery()` for high-frequency queries.

Red Flag Answer: Increasing SQL command timeout without resolving the underlying missing index or unindexed query scan.

Follow-up Question: "How does EF Core 10 improve LINQ translation for non-primitive collections?"

Q18. Bulk Update: Set IsActive = false for all products in a category. Most efficient way?

[Mid]

Great Answer Strategy:

- Use `ExecuteUpdateAsync` (EF Core 7+): `db.Products.Where(p => p.CategoryId == 5).ExecuteUpdateAsync(s => s.SetProperty(p => p.IsActive, false))`.
- Generates a single direct SQL UPDATE statement without loading entities into memory.
- Caveat: Bypasses Change Tracker, interceptors, and soft delete events.

Red Flag Answer: Loading all entities into memory and updating them in a foreach loop before calling `SaveChangesAsync()`.

Follow-up Question: "How do you manage audit logging when using `ExecuteUpdateAsync`?"

4. AUTHENTICATION & SECURITY

5 Questions

Q19. Your JWT is valid for 15 mins. A user's role changes. The token still has Admin claim. Now what?

[Senior]

Great Answer Strategy:

- Issue short-lived access tokens (5-15 min) paired with refresh tokens; re-check roles in DB during refresh.
- For highly sensitive/destructive operations, check user roles against DB/cache in handler instead of relying solely on JWT claims.
- Maintain a Redis token blacklist/revocation list for instant access termination when required.

Red Flag Answer: Claiming JWTs cannot be invalidated so nothing can be done.

Follow-up Question: "How do you structure refresh token rotation to detect token theft?"

Q20. API called from React SPA on a different domain. What CORS configuration?

[Mid]

Great Answer Strategy:

- Explicitly define policy with `.WithOrigins('https://app.example.com')` and allowed methods/headers.
- Use `.AllowCredentials()` if sending cookies/authorization headers across origins.
- CRITICAL: Never combine `AllowAnyOrigin (*)` with `AllowCredentials` (browsers reject this).
- Place `UseCors()` before `UseAuthentication()` in middleware pipeline.

Red Flag Answer: Using `AllowAnyOrigin()` and `AllowAnyHeader()` in production environments.

Follow-up Question: "Why does Postman succeed on CORS requests that fail in web browsers?"

Q21. How do you securely store connection strings across environments?

[Senior]

Great Answer Strategy:

- Development: Use Secret Manager (dotnet user-secrets).
- Production: Use Azure Key Vault, AWS Secrets Manager, or Kubernetes Secrets.
- Never commit connection strings or API keys to appsettings.json or source control.

Red Flag Answer: Adding appsettings.json containing passwords to source control or relying on gitignore alone.

Follow-up Question: "How does the Options Pattern with IOptionsMonitor handle secret rotation?"

Q22. What's the difference between Authentication and Authorization?

[Junior]

Great Answer Strategy:

- Authentication = 'Who are you?' (verifying identity, returns 401 Unauthorized if invalid/missing).
- Authorization = 'What are you allowed to do?' (checking permissions, returns 403 Forbidden if prohibited).
- Note: 401 HTTP status is technically 'Unauthenticated', while 403 is 'Unauthorized/Forbidden'.

Red Flag Answer: Confusing 401 and 403 status codes or treating identity and permissions as identical.

Follow-up Question: "How do requirement-based custom authorization handlers work in ASP.NET Core?"

Q23. How do you prevent Mass Assignment (Over-Posting) attacks?

[Senior]

Great Answer Strategy:

- Always bind request endpoints to dedicated Request DTOs, never directly to EF Core Data Model entities.
- Explicitly map allowed fields from DTO to entity manually or via mapping utilities.
- Set privileged properties (IsAdmin, AccountBalance) explicitly in backend business logic.

Red Flag Answer: Accepting DB Entity classes directly in action parameters (e.g. `MapPost('/users', (User user) => ...)`).

Follow-up Question: "Why is the [Bind] attribute inferior to dedicated request DTOs?"

5. PERFORMANCE & CACHING

5 Questions

Q24. Your API endpoint takes 800ms. How do you diagnose and fix it?

[Senior]

Great Answer Strategy:

- Profile first: Enable OpenTelemetry tracing or APM logging to identify the slow component.
- Database: Check SQL execution time & N+1 queries using EF Core logging.
- External Calls: Parallelize independent downstream HTTP calls via `Task.WhenAll`.
- Payload: Reduce response size with DTO projections and response compression.

Red Flag Answer: Adding caching blindly before profiling and pinpointing the exact bottleneck.

Follow-up Question: "How do you measure P95 and P99 latency distribution using OpenTelemetry?"

Q25. What's the difference between Response Caching, Output Caching, and HybridCache?

[Mid]

Great Answer Strategy:

- Response Caching: Sets HTTP headers (Cache-Control) for client/CDN caching.
- Output Caching (.NET 7+): Server-side endpoint caching supporting tag-based eviction (.EvictByTagAsync).
- HybridCache (.NET 9+): Multi-tier (L1 in-memory + L2 distributed Redis) with built-in cache stampede protection.

Red Flag Answer: Using IMemoryCache everywhere without eviction policies or stampede guardrails.

Follow-up Question: "What is a cache stampede and how does HybridCache's request coalescing prevent it?"

Q26. Three independent async operations, each 200ms. How do you optimize?

[Senior]

Great Answer Strategy:

- Run concurrently using Task.WhenAll(task1, task2, task3) -> Reduces total execution time from ~600ms to ~200ms.
- Important boundary: Avoid running parallel DB operations on the SAME DbContext instance (DbContext is not thread-safe; use separate scope factories).

Red Flag Answer: Awaiting independent async tasks sequentially one after another.

Follow-up Question: "How do you handle partial failures when executing multiple tasks in Task.WhenAll?"

Q27. What common async/await mistakes hurt API performance?

[Mid]

Great Answer Strategy:

1. Wrapping simple task returns in async/await unnecessarily (adds state machine overhead).
2. Blocking with .Result or .Wait() causing thread pool starvation and deadlocks.
3. Using async void (exceptions unhandled, crashes worker process).
4. Fire-and-forget without error monitoring/BackgroundService.

Red Flag Answer: Using Task.Run inside Web API endpoints to offload I/O tasks.

Follow-up Question: "Why is ConfigureAwait(false) less critical in ASP.NET Core than in desktop UI frameworks?"

Q28. What does async/await actually do?

[Junior]

Great Answer Strategy:

- Yields worker threads back to the thread pool while awaiting I/O operations (database, disk, network).
- Increases overall server throughput and concurrency capacity, allowing handling of thousands of concurrent requests.
- Does NOT make an individual request execute faster (latency remains identical).

Red Flag Answer: Stating that async/await speeds up CPU calculation speed or decreases single request latency.

Follow-up Question: "How does the state machine generated by C# async methods operate?"

6. ARCHITECTURE & SYSTEM DESIGN

4 Questions

Q29. Should you use the Repository Pattern with EF Core?

[Senior]

Great Answer Strategy:

- DbContext already acts as a Repository + Unit of Work implementation; generic IRepository adds redundant abstraction.
- Prefer focused Query Services or CQRS handlers using DbContext directly for projections.
- Use repositories only when hiding multi-source data retrieval or enforcing strict DDD aggregate boundaries.

Red Flag Answer: Building generic IRepository with methods like GetAll() that return IQueryable.

Follow-up Question: "How do Query Services simplify unit testing without generic repositories?"

Q30. Explain CQRS. When is it overkill?

[Senior]

Great Answer Strategy:

- CQRS (Command Query Responsibility Segregation) separates Read models from Write models.
- Essential when: High read-to-write ratios (100:1), complex domain logic vs simple queries, or separate event-driven data stores.
- Overkill for basic CRUD applications with identical read and write representations.

Red Flag Answer: Assuming CQRS mandates MediatR, Event Sourcing, and multiple databases.

Follow-up Question: "How can you implement CQRS without introducing third-party mediator libraries?"

Q31. What's Vertical Slice Architecture vs Clean Architecture?

[Mid]

Great Answer Strategy:

- Clean Architecture organizes code by technical layers (Domain, Application, Infrastructure, API).
- Vertical Slice Architecture organizes code by business features; each feature slice contains its own endpoint, request handler, and DTOs.
- Vertical Slices minimize cross-project coupling for API feature additions.

Red Flag Answer: Insisting Clean Architecture is superior for simple CRUD services.

Follow-up Question: "How do you share common domain rules across vertical slices?"

Q32. How do you decide between a Monolith and Microservices?

[Mid]

Great Answer Strategy:

- Default to Monolith or Modular Monolith for small/mid teams, fast domain iteration, and lower operational overhead.
- Adopt Microservices only for independent team deployments, isolated scaling bottlenecks, or polyglot requirements.
- Modular Monoliths enforce clean boundaries per module, allowing easy microservice extraction later if necessary.

Red Flag Answer: Choosing Microservices prematurely without DevOps automation or distributed system tracing.

Follow-up Question: "How do you enforce module boundary isolation in a .NET Modular Monolith?"

7. TESTING

3 Questions

Q33. How do you write Integration Tests for an API with DB & external services?

[Senior]

Great Answer Strategy:

- Use WebApplicationFactory for in-memory HTTP test host pipeline execution.
- Use Testcontainers for real database testing (catches SQL syntax & constraint errors that InMemory DBs miss).
- Use WireMock.NET to mock third-party external HTTP services deterministically.

Red Flag Answer: Relying on EF Core InMemory Database provider for integration testing SQL features.

Follow-up Question: "How do you speed up Testcontainers spin-up time during local test runs?"

Q34. What should you test with Unit Tests vs Integration Tests?

[Mid]

Great Answer Strategy:

- Unit Tests: Business logic, domain entity rules, custom algorithms, validation logic.
- Integration Tests: Full HTTP pipeline, EF Core database queries, authorization rules, middleware ordering.
- Recommended ratio: ~70% Integration tests, ~20% Unit tests, ~10% Architecture tests.

Red Flag Answer: Writing unit tests for LINQ-to-Entities queries against mocked DbSets.

Follow-up Question: "How do NetArchTest architecture tests enforce project dependency rules?"

Q35. How do you test an endpoint that requires authentication?

[Senior]

Great Answer Strategy:

- Create a custom TestAuthenticationHandler in test project replacing JWT bearer auth.
- Inject mock ClaimsPrincipal with required user ID and roles for test scenarios.
- CRITICAL: Include tests validating that unauthenticated requests return 401 and missing roles return 403.

Red Flag Answer: Removing [Authorize] attributes or bypassing security conditionally in the production codebase during testing.

Follow-up Question: "How do you verify token expiration logic in integration tests?"

8. PRODUCTION READINESS

4 Questions

Q36. How do you implement Structured Logging?

[Senior]

Great Answer Strategy:

- Use message templates with named placeholders: `logger.LogInformation("Order {OrderId} processed for {UserId}", orderId, userId).`
- Enables log aggregators (Seq, Elastic, Loki) to index properties for structured querying.
- Integrate Serilog with correlation IDs and `UseSerilogRequestLogging()` for single summary HTTP log entries.

Red Flag Answer: Using string interpolation `logger.LogInformation($"Order {orderId}")` which destroys searchable properties.

Follow-up Question: "How do you sanitize PII (Personally Identifiable Information) from structured logs?"

Q37. How do you implement Health Checks?

[Mid]

Great Answer Strategy:

- Use `Microsoft.Extensions.Diagnostics.HealthChecks` to add DB and Redis health checks.
- Separate Liveness (`/health/live` - process responsive) from Readiness (`/health/ready` - DB/cache connected).
- Kubernetes restarts containers failing liveness and removes traffic from containers failing readiness.

Red Flag Answer: Testing heavy DB queries inside liveness probes, causing container restart loops.

Follow-up Question: "How do custom `HealthCheckUI` dashboards display endpoint health status?"

Q38. How do you containerize a .NET API?

[Mid]

Great Answer Strategy:

- Use Multi-stage Dockerfiles (`mcr.microsoft.com/dotnet/sdk` for build, `dotnet/aspnet` for runtime).
- Use Chiseled runtime images (.NET 8+) for minimal image size (~30MB) and reduced attack surface.
- Run container under non-root USER (`$APP_UID`) on default port 8080.

Red Flag Answer: Shipping full SDK images to production or running containers as root user.

Follow-up Question: "How does layer caching order in Dockerfiles accelerate CI/CD build speeds?"

Q39. How do you monitor your API in production?

[Senior]

Great Answer Strategy:

- Implement the 3 Pillars of Observability: Logs, Metrics, and Traces via OpenTelemetry.
- Export metrics/traces via OTLP to Prometheus/Grafana or Datadog.
- Set proactive alerts on HTTP 5xx error rate spikes (>1%) and P95 latency SLA breaches.

Red Flag Answer: Relying strictly on manual log inspection after users report production outages.

Follow-up Question: "How does W3C `TraceContext` propagate correlation IDs across microservice calls?"

9. MODERN .NET & C# FEATURES

6 Questions

Q40. What are Primary Constructors in C# 12+?

[Mid]

Great Answer Strategy:

- Allows declaring constructor parameters directly on the class declaration line.
- Eliminates private readonly backing field boilerplate in Dependency Injection service classes.
- Parameters are in scope throughout class body.

Red Flag Answer: Confusing class primary constructors with record positional syntax.

Follow-up Question: "What happens when you capture primary constructor parameters in mutable fields?"

Q41. What are Records and when would you use them for DTOs?

[Mid]

Great Answer Strategy:

- Records provide built-in value-based equality, non-destructive mutation (with expression), and immutability.
- Ideal for API DTOs, request/response models, and domain events.
- Use standard classes for EF Core tracked entities requiring reference equality and state updates.

Red Flag Answer: Treating records as simple syntax sugar without understanding value equality semantics.

Follow-up Question: "How does record inheritance differ from standard class inheritance?"

Q42. What are Minimal APIs and why choose them over Controllers?

[Mid]

Great Answer Strategy:

- Minimal APIs declare route endpoints directly on WebApplication with lower memory/CPU footprint.
- Eliminates MVC controller reflection overhead; offers full Native AOT compilation support.
- TypedResults return types provide compile-time OpenAPI schema generation.

Red Flag Answer: Claiming Controllers are obsolete or forcing Minimal APIs without grouping route handlers.

Follow-up Question: "How do Route Groups (.MapGroup) organize Minimal API endpoints?"

Q43. What's new in .NET 10 for Web API Development?

[Senior]

Great Answer Strategy:

- Built-in OpenAPI generation replacing Swashbuckle.
- HybridCache with request coalescing stampede protection.
- Enhanced ExecuteUpdateAsync / ExecuteDeleteAsync LINQ capabilities.
- Built-in rate limiting and container publishing improvements (dotnet publish /t:PublishContainer).

Red Flag Answer: Referencing obsolete .NET 6 packages or third-party workarounds for native features.

Follow-up Question: "How does native OpenAPI generation in .NET 10 handle schema generation?"

Q44. What is CancellationToken and why should every async endpoint accept one?

[Mid]

Great Answer Strategy:

- Signals when client disconnects or browser aborts request during processing.
- Propagate CancellationToken to EF Core queries, HttpClient calls, and I/O tasks to terminate wasted server work.
- Saves database connection pool threads and server CPU on cancelled requests.

Red Flag Answer: Omitting CancellationToken from async database queries in production endpoints.

Follow-up Question: "How does Kestrel auto-bind CancellationToken parameters in route handlers?"

Q45. What's the 'field' keyword in C# 14?

[Senior]

Great Answer Strategy:

- Allows accessing the compiler-generated backing field directly inside property getters/setters.
- Eliminates manual private backing field declarations (_fieldName) when adding validation logic to properties.
- Example: public string Name { get; set => field = value ?? throw new ArgumentNullException(); }

Red Flag Answer: Declaring redundant backing fields in C# 14 property accessor implementations.

Follow-up Question: "How does the field keyword interact with init-only properties?"

- 1. Reciting Definitions Instead of Experience:** Generic textbook definitions signal lack of hands-on work. Share real debugging scenarios and trade-offs.
- 2. Saying 'It Depends' Without a Decision Framework:** Always follow 'it depends' with your default recommendation and explicit criteria for when to deviate.
- 3. Staying Stuck in Obsolete .NET Patterns:** Referencing Swashbuckle, .NET 6 limits, or obsolete libraries signals you stopped learning.
- 4. Lacking Production War Stories:** Interviewers look for signals like performance profiling, incident response, and connection pool management.
- 5. Skipping the 'Why' Behind Architectural Choices:** Explain the exact reasoning, trade-offs, and alternative approaches considered.